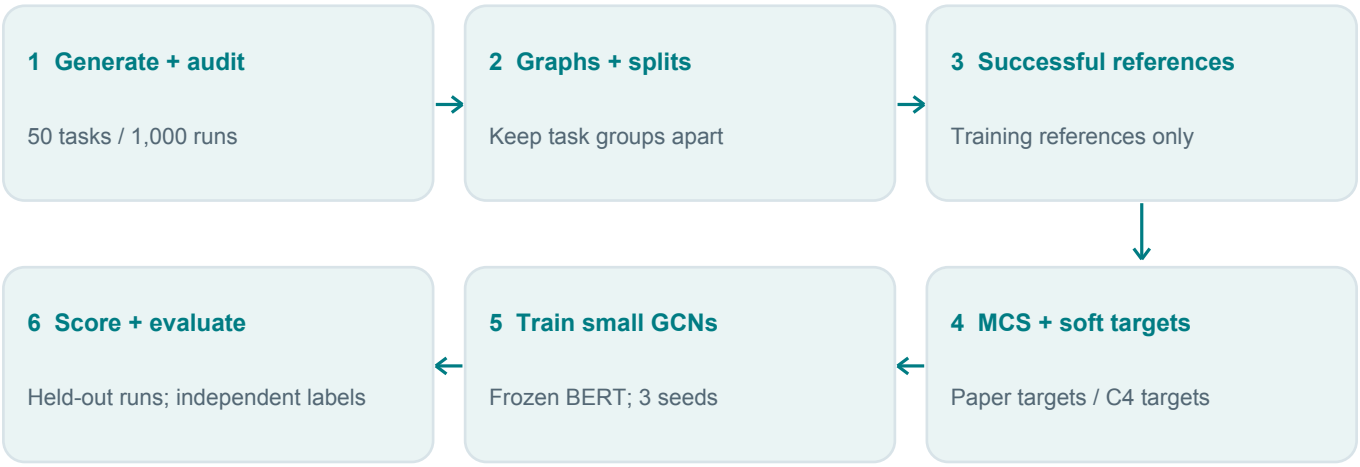
Stepwise confidence for airline agents

A complete project walkthrough | CSE598 Team 11

What we are solving, what each method does, why each step is needed, and what remains before the fresh experiment is complete.



The central question: can we identify suspicious actions inside a failed agent run, instead of receiving only a final pass/fail result?

Completed	In progress / next
1,000 trajectories; all 50 tasks	MCS targets: 271/860 in the worker snapshot
NIBS and simple baselines	Fresh GCN training follows MCS
Dataset/results pushed to GitHub	Independent evaluation-label review remains pending

This guide describes the current Qwen/ASU experiment. Older Groq experiments are historical evidence and are identified separately.

Prepared: October 29, 2026, 12:39 PM Arizona time. Progress is a dated snapshot, not a live display.

Scope: reproduce the paper's method on tau2-bench Airline and evaluate C4 as a project extension. This is not an exact reproduction of the original math/QA experiments.

How to read this guide

Start with the problem; follow the pipeline; use the checklists and file map later.

Pages	What you will understand
3–4	The problem, Task 25, the dataset, and the model roles
5–6	The end-to-end workflow and the actual graph representation
7–9	NIBS, MCS, and exactly how C4 differs
10–11	Why 1,000 becomes 860 or 911, and what balance means
12–13	Held-out evaluation, GCN training, and inference
14–16	Labels, metrics, current results, and replication limits
17–18	Course phases, next actions, GitHub, files and monitoring
19–20	Glossary, sources and all 50 task outcome counts

Five numbers to remember

- **1,000:** generated and graded trajectories, across all 50 tasks.
- **680 / 320:** official benchmark passes and failures. This is a 68% agent task pass rate.
- **860:** trajectories eligible to supply training/validation MCS targets under our reference rules.
- **911:** trajectories eligible for the common NIBS baseline comparison.
- **24:** planned GCN checkpoints: 2 methods × 4 held-out folds × 3 seeds.

Pass rate, MCS target, confidence score and error-detection accuracy are four different quantities. The guide keeps them separate.

1. What are we solving?

A final failure reward does not tell an engineer which action to investigate.

An airline assistant may read a user profile, inspect an existing reservation, search flights, choose payment methods, book a ticket and send a message. A final reward can say that the task failed, but it does not identify the wrong passenger, incorrect payment choice, missing action or misleading message.

The output we want

Give each analysed agent tool call or message a score. Rank low-confidence steps for inspection. The intended users are engineers and researchers debugging tool-using agents.

A concrete case: what Task 25 actually asks

Task 25 asks the assistant to book a friend on an itinerary matching an existing reservation. It must use the friend's correct profile details, book only that friend, choose payment according to a $400 condition, avoid baggage and insurance, and explain any remaining certificate balance.

Calling book_reservation is not enough: the passenger, itinerary and payment must be right. This is why matching only the tool name can miss an important mistake.

In the fresh Qwen collection, Task 25 passed all 20 runs. It is useful for a successful walkthrough but cannot provide a failed-run localization example in this collection. Earlier Groq Task 25 outcomes are different data.

What the system establishes

• A low score means weaker agreement with successful patterns or the learned targets. It is a diagnostic signal, not proof of the causal failure.

• The current pipeline analyses recorded executions. Live intervention, automatic repair and safer actions are possible future applications; they are not demonstrated by the current results.

Milestone 1 began with tool_sim NIBS. The project now adds semantic paper NIBS, the PDF-style learned GIBS predictor and C4 supervision.

2. What data and models do we use?

A task is a scenario; a trajectory is one complete attempt at that scenario.

Term	Meaning in this project
Task	One fixed airline customer-service scenario. Task IDs are 0–49.
Trajectory / run	One assistant–customer execution, including tool calls, results and final reward.
Step	One analysed agent tool call or message. User turns provide context.
Collection	50 tasks × 20 attempts = 1,000 distinct graded trajectories.

Four model roles; only the small predictor is trained

Component	What it does	Training here?
Qwen3.8-27B	ASU-hosted assistant and customer simulator; generates the executions.	No
BERT base	Converts operation/result text into fixed numerical features.	Frozen
DeBERTa-large-MNLI	Estimates entailment for graph matching.	Frozen
GCN + prediction layers	Learns to predict per-step MCS-derived targets.	Yes

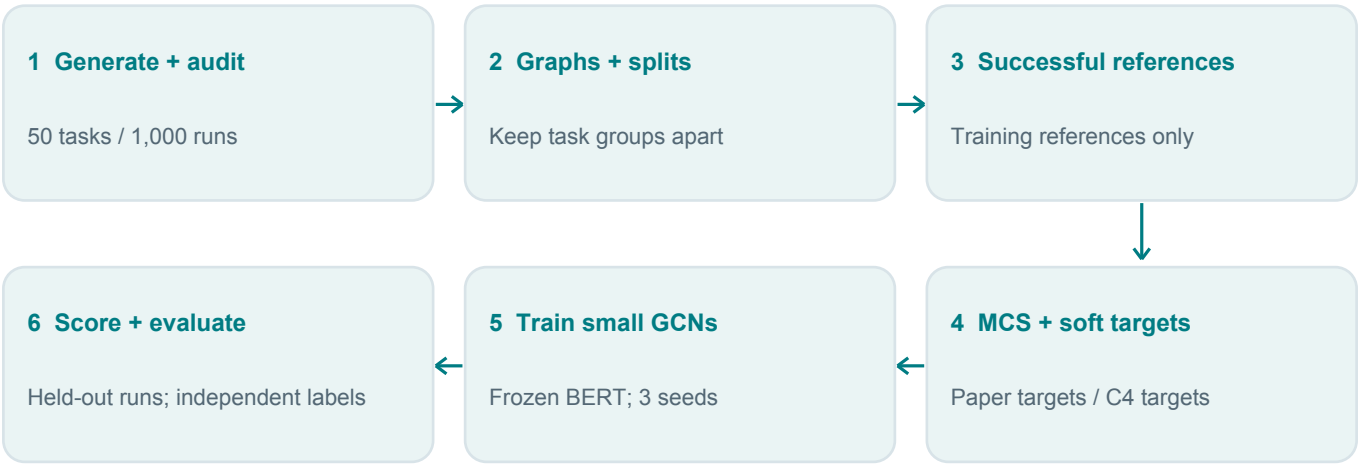
The assistant temperature is 1.0; the user simulator temperature is 0.0. Each task has 20 runs. The collection uses seed 300, a 200-step limit, a 10-error limit, and explicitly disables reasoning output and parallel tool calls.

The source is a real, published benchmark with simulated customers and sandboxed airline state. It is not private production airline traffic. Collection used ASU's no-charge hosted model service; graph matching uses Sol and predictor fitting runs locally.

All 1,000 final records are graded and parse into graphs. Infrastructure failures were kept in separate pre-resume snapshots and retried. Thirty passing records have missing-expected-tool review flags; official reward alone does not certify every step.

3. The complete workflow

Each stage produces a concrete input for the next stage.



Stage	Why we do it	Output
Generate and audit	Capture different executions; exclude unfinished infrastructure attempts from the final graded collection.	Recorded, distinct runs
Build graphs and split	Represent operations/results and prevent held-out task leakage.	Graphs and fixed folds
Choose successful references	Define examples of benchmark-successful behaviour for each task.	Reference IDs
Compute MCS and average matches	Turn structural agreement into per-step supervision.	Soft target masks
Train GIBS / C4 predictors	Learn graph patterns so new runs can be scored without MCS at prediction time.	Saved GCN checkpoints
Evaluate and review	Check whether scores identify independently labelled errors better than baselines.	Metrics, coverage and review sheets

NIBS takes a shorter route: query steps + successful reference steps → similarities → scores. It does not require MCS or predictor training.

Do not use agreement with an MCS teacher as the sole proof that the teacher is correct. Evaluation needs step labels obtained independently of that matching target.

4. What is a trajectory graph?

Observed operations and results become connected events; hidden reasoning is not available.

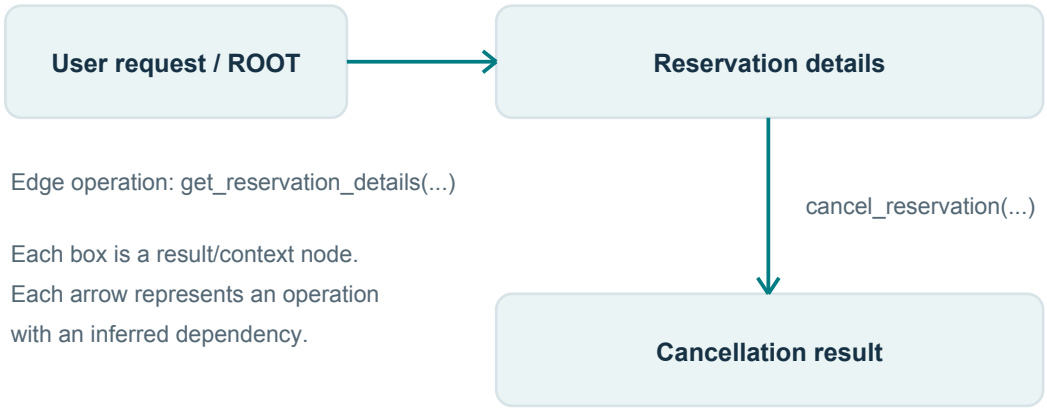


Illustration only: this diagram is not a claimed recorded benchmark execution. It shows how an operation produces a result and depends on earlier information.

The graph used by the current fresh experiment

• Tool calls produce result nodes; operation text contains the call. Agent messages and user turns are represented as events. ROOT supplies the initial-state connection.

• The current comparison calls projection(..., structure="original"), which uses the **paper_literal** projection. Dependencies are inferred from values used in earlier observed events; dependency-free events attach to ROOT.

• The projection does not automatically add an edge from every immediately preceding turn. User nodes are context only and do not receive confidence scores.

Why this matters: a graph can distinguish structurally connected operations from unrelated steps that merely sound alike. However, the inferred graph is a heuristic representation, not verified causal reasoning.

> The repository also contains an event_context projection and a separate state-aware graph ablation. Neither should be silently described as the fresh experiment's active graph. For current C4 targets, "context support" means its actual incoming dependency/root edges.

The predictor pools information across the recorded graph. Current results are therefore retrospective diagnostics, not a demonstrated prefix-only online safety system.

5. NIBS and the tool_sim baseline

NIBS directly compares steps; it learns no model parameters.

For one query step, compare its operation text with all agent-operation steps in each successful reference. Paper semantic NIBS uses frozen BERT features and cosine similarity.

• **Cos-Max:** take the best-matching step within each reference, then average those maxima across references.

• **Cos-Mean:** average similarity over all steps within each reference, then average those means across references.

• Each reference trajectory receives equal weight, even if its number of steps differs.

A small calculation

Reference	Two illustrative similarities	Maximum	Mean
R1	0.8, 0.4	0.8	0.6
R2	0.7, 0.5	0.7	0.6
R3	0.9, 0.3	0.9	0.6

The query step's Cos-Max score is (0.8 + 0.7 + 0.9) / 3 = **0.8**. Its Cos-Mean score is **0.6**. These are made-up numbers to explain aggregation, not experimental results.

Where tool_sim fits

The original agent baseline compares tool names and argument values directly. It is useful and inexpensive, but it is not the paper's semantic NIBS. It cannot by itself verify that a correct-looking call is justified by the conversation or that a message is truthful.

Raw cosine can be negative and is not automatically a probability. Do not describe a NIBS score of 0.8 as "80% certainly correct." NIBS also needs successful same-task references at evaluation time.

6. What is MCS, and why first?

MCS means Maximum Common Subgraph; our implementation uses a heuristic search.

MCS finds a large matching pattern of nodes and edges between a query graph and a successful reference graph.

• Use DeBERTa-large-MNLI to score whether operation/result texts support one another. The matching similarity is the maximum of both entailment directions, following the released implementation.

• Use node and edge thresholds of 0.7. Gather compatible matches, try the top 10 seeds and expand them while preserving a consistent node mapping.

• Save the matched node map, edge map and the query's per-step indicators. The heuristic does not guarantee the mathematically largest possible subgraph.

• Repeat against 3–5 selected successful references, then average the indicators to get one soft target per analysed step.

How the paper-style target is calculated

```
target(step) = matched result-node count / reference count
Example: [1, 1, 0, 1, 1] → 4/5 → 0.8
```

A target of 0.8 means the result node appeared in four of five matched structures. It is agreement-based supervision, not an independently verified correctness label.

Why this stage is slow

Many query/reference edge and node text pairs need NLI inference. Results are cached, and completed pair mappings and target files can be reused after interruptions. The expensive teacher stage runs on Sol's A100 GPU.

MCS is needed before this GIBS training procedure because it supplies the target masks. Once the predictor is trained, ordinary GCN inference does not recompute MCS.

7. How C4 differs from paper GIBS

C4 is the project's variant; its behaviour is defined by the implementation.

Part	Paper-style teacher	C4 teacher
Semantic matching	DeBERTa entailment	Same model
Event kinds	Cross-kind candidates allowed	Same-kind matching required
Critical tool fields	No structured-field rejection	Reject listed argument/result conflicts
Long inputs	Author-style joined-text truncation	Symmetric token-budget truncation
Matched structure	Single connected component	Multiple compatible components
Step target	Result node appears in MCS	Every incoming edge of the result node is matched

What "every incoming edge" means

If a result node has two incoming dependencies, C4 supports that step only when both corresponding edges match the successful reference. Paper node membership can count a result node that entered the MCS through just one matched relationship.

What the field checks actually inspect

They check listed decision-critical values such as reservation ID, selected flight/date, passenger details, payment fields and error status. For example, two cancellation calls with different reservation IDs can be rejected even if their text is semantically similar. Free-text summaries and unlisted fields are generally left to NLI.

> Current C4 uses the same paper_literal graphs as paper GIBS. It does not automatically verify every prior user turn. Its teacher checks are not hard constraints enforced by the trained predictor at inference.

Both learned methods use the same GCN architecture and training setup. Changing the teacher lets us test whether these stricter matches improve useful error detection. They may also reject valid alternative behaviour; improvement must be measured.

8. Why 1,000 becomes 860 or 911

The three numbers refer to different populations, not missing data.

Number	Meaning	Reference access
1,000	All generated/graded runs; all are intended to receive learned held-out predictions.	No references needed at learned inference
860	Eligible training/validation queries for MCS targets.	Successful training-partition references only
911	Eligible queries for NIBS and the common method comparison.	Successful references from the evaluation reference bank

The 860 calculation

Our prototype requires at least three other successful training references of the same task and caps the selected bank at five. The query's entire exact-content class is excluded. The minimum and cap are project choices, not specified requirements of the paper.

Tasks	Passes per task, out of 20	Successful training-bank runs
7	0	0
14, 29, 44	2	1
20, 21, 32	3	2

Each of these seven tasks contributes 20 queries without enough training references: **1,000 − (7 × 20) = 860**. The 140 excluded queries remain in the dataset and are still scored by held-out GCNs.

The 911 calculation

NIBS uses the larger successful evaluation-reference bank: tasks 7, 14, 29 and 44 exclude 80 runs. Tasks 20, 21 and 32 each have three successes: their successful queries have only two other successes, excluding nine more. Thus **1,000 − 80 − 9 = 911**.

> The 860 include targets shared across fold-specific training and validation subsets. They are not 860 examples trained together in every fold. Never invent scores for missing-reference NIBS cases.

9. What balanced pass/fail means

Equal outcome counts are different from useful step-level evaluation.

Phrase	Meaning
Overall 50:50 balance	500 successful and 500 failed trajectories out of 1,000.
Per-task 50:50 balance	10 successful and 10 failed runs for each task's 20 attempts.
Current collection	680 successful / 320 failed. No forced outcome balancing.
Balanced training	A separate sampling/weighting choice affecting training influence; actual outcomes do not change.

What the paper says

The paper samples 20 traces per question at temperature 1.0 to trade off accuracy and diversity. Appendix D.3 describes training on a mixture of correct and incorrect trajectories. It does not specify a required 50:50 pass/fail ratio or 10/10 per question.

What our temperatures do

We already used assistant temperature 1.0 and user temperature 0.0. This encourages assistant variation while limiting simulator randomness. It does not guarantee mixed outcomes: 12 fresh tasks passed all runs and one failed all runs.

• All-pass tasks can still supply successful MCS references; they lack failed runs for failed-run localization.

• All-fail tasks cannot provide their own successful references, but a GCN trained on other tasks can still score them.

• The suggested 3–17 passes out of 20 identifies 24 mixed-outcome tasks. It is an optional additional analysis; the current experiment does not filter to them.

A failed trajectory can contain many correct steps. Balancing whole-run outcomes does not balance correct/incorrect steps. A balanced-training experiment would be a separately reported control, not a required correction to the paper method.

10. How do we avoid training on the test?

Four task-group folds let every trajectory be held out once per seed.

We group tasks sharing user/reservation identifiers or identical scenario text. Entire groups are kept together. This automatic grouping is not human certification that all related scenario families are independent.

Held-out fold	Test trajectories	Role of other task groups
Fold 0	260	Training + validation
Fold 1	260	Training + validation
Fold 2	240	Training + validation
Fold 3	240	Training + validation
Total	1,000	Each query appears in exactly one test fold per seed

The isolation rules

• The model scoring a trajectory excludes its entire task group from fitting and early stopping.

• Within fitting tasks, approximately 20% per task/outcome stratum is reserved for validation. This preserves outcome strata; it is not a 50:50 sampler.

• Training and validation MCS targets use successful references from the fitting partition only. Reused task-local targets are checked against each fold's actual reference pool.

• Frozen BERT features may be cached across all runs because the encoder learns no parameters from them. Gold actions and rewards are not prediction features.

A reference baseline and a learned predictor have different access: NIBS receives known-successful same-task examples during scoring; the learned GCN receives the query graph only. This distinction must appear in the results report.

Three seeds (7, 17, 27) measure variation from initialization and optimization. Report mean and standard deviation, not just the best seed.

11. What exactly is GCN training?

A small learned predictor estimates MCS-derived support for each agent event.

BERT converts result text and operation text into frozen 768-dimensional features. The GCN passes information across graph dependencies. The prediction layers combine a local result representation, the operation representation and pooled graph context to produce a score for each agent event.

Training choice	Current implementation
Encoder	Frozen BERT CLS features; maximum 256 tokens per text
Graph encoder	Two GCN layers; hidden size 128; dropout 0.1
Objective	Paper Equation 6: mask entropy + lambda × soft binary cross-entropy; lambda = 1
Optimization	Adam, learning rate 0.001; batches of eight query graphs
Stopping	Up to 150 epochs; validation-loss early stopping, patience 20
Repeated fits	Two target methods × four folds × three seeds = 24 checkpoints

The cross-entropy term keeps predicted scores close to the MCS soft targets. The entropy term encourages decisive masks. A learned mask is not evidence that we discovered a causally minimal explanation.

Training versus inference

• **Training:** query graph + stored target mask → update GCN/prediction weights. Validation chooses the checkpoint.

• **Inference:** recorded query graph + saved checkpoint → scores in [0,1]. No reference runs, MCS, final reward, gold actions or step labels are supplied to the forward pass.

Qwen, BERT and DeBERTa are not being fine-tuned. The fresh GCN fits have not started in this snapshot; they are scheduled after MCS completes. Existing historical checkpoints are a separate completed prototype.

12. How do we measure whether it works?

Use independent correctness labels, then compare scores on the same cases.

Two different kinds of labels

• **Teacher targets:** MCS agreement fractions. They supervise GCN training.

• **Evaluation labels:** independent judgements of whether a step is correct, incorrect or unresolved. They assess whether the scores are useful.

The current automatic verifier covers selected tool actions and fields; many steps remain unverified. A benchmark-successful run is not automatically labelled correct at every step. Unknown and attempted-incorrect steps are excluded from binary correctness metrics.

Metric	Question answered	Current status
AUROC	Do incorrect steps rank lower than correct steps?	Implemented
Average precision / PR	How well are rare errors retrieved?	Per-task AP implemented; define PR convention when comparing paper metrics
First-error Top-1 / Top-3 / MRR	How highly is the first verifier-labelled error ranked?	Implemented; localization proxy
ACC@80%	After retaining the highest-confidence 80%, what fraction is correct?	Not yet implemented
ECE	Do predicted confidence levels agree with correctness frequencies?	Not yet implemented

AUROC requires both label classes; undefined task metrics must be N/A. Unverified steps can still compete in error-localization rankings. A missed action may have no executed step to rank.

Paper D.3 uses GPT-4o step judgements. Our narrow rule labels share structured-field assumptions with the matcher; an automatic-label gain can favour those assumptions. Independent review is needed for wrong arguments, messages, omissions and reward flags. Raw cosine is not a probability; define any calibration mapping using validation data only.

13. What results do we have now?

These are fresh Qwen baseline results; the learned comparison is pending.

Completed method	Tool macro AUROC	First-error Top-1
Paper NIBS Max	0.931	57.2%
Paper NIBS Mean	0.724	71.7%
tool_sim baseline	0.994	59.0%
Random expectation	0.500	14.3%
Last-step-first	0.726	66.5%

Population: 911 reference-eligible trajectories. Tool AUROC averages 26 tasks containing both automatic correctness classes. Top-1 covers 173 failed trajectories with a labelled tool error. These denominators are smaller than all 1,000 runs.

NIBS Max has stronger AUROC than Mean here, but Mean has better first-error localization. These metrics answer different questions. The tool_sim AUROC of 0.994 is not a 99.4% agent pass rate and does not establish subtle-error detection.

What is still unknown

We do not yet have fresh trained GIBS/C4 scores, a completed head-to-head learned comparison, or independent human-validated accuracy. Most of the 14,145 analysed agent steps are unverified: 11,380 unverified, 2,489 labelled correct, 267 labelled incorrect and 9 attempted-incorrect.

Dated execution snapshot

Stage	Status
Collection + graph compatibility	Complete: 1,000 distinct graded runs; zero unresolved infrastructure errors
NIBS baseline and GitHub upload	Complete
MCS teacher generation	271 / 860 query targets in the monitor snapshot; Sol job 66924222 running
Fresh GCN fitting + learned evaluation	Pending target completion
Independent review; ACC@80%; ECE	Pending

Snapshot checked: Oct 29, 2026, 12:39 PM Arizona. Counts will advance after this PDF is generated. Read worker_state.json for the latest monitor state.

14. Are we replicating or extending?

The accurate description is a method transfer plus an agent-specific extension.

Aspect	Paper / original setup	Our current study
Problem domain	Math and multi-hop QA	Tool-using Airline assistant
Repeated samples	20 traces at temperature 1.0	20 runs/task; assistant 1.0, user 0.0
Graph construction	Structured reasoning graphs	Observed event/value-dependency adapter
GIBS architecture	PDF describes a two-layer GCN	Two-layer GCN; explicit fusion choices
Training supervision	Consensus from correct graphs	Benchmark-successful training references; minimum 3, cap 5
Scale	2,000 training graphs; 10,000 evaluated trajectories/dataset	1,000 total runs; four-fold evaluation
Step evaluation	GPT-4o judgements	Partial rule verifier; human review pending
C4	Not the paper's named method	Our field/context/multi-component teacher variant

The paper's stated GCN and the released script are not identical: the release's active predictor uses MLPs/mean pooling. The repository contains separately labelled controls; our current primary learned comparison follows the PDF-style GCN. It is not a bit-for-bit replication of that release.

A suitable project claim: "We implement the paper's stepwise-confidence methods on tau2-bench Airline, compare against step-level baselines, and evaluate C4 as an agent-specific supervision variant."

A claim not yet supported: "We reproduced the paper's published benchmark accuracy" or "C4 is better." New data, graph construction, labels, scale, reference access and runtime versions must be disclosed.

Runtime: Sol teacher generation uses FP32 on an A100, Torch 2.11.0 and Transformers 4.57.6; local fitting uses Torch 2.5.1 and Transformers 4.51.3. CPU/CUDA smoke checks passed. Runtime JSON files record hardware and versions.

15. Course phases and your next actions

The expanded experiment is in Phase 2; completion needs results and defensible evaluation.

Course phase	Project evidence / remaining work
Phase 1: discovery	Milestone 1 records a stakeholder meeting with Xiaoou Liu on September 23, 2026, scoped problem and proposed metrics. Later stakeholder feedback is not verified here.
Phase 2: prototype	Historical prototype exists; fresh 1,000-run collection and NIBS evaluation are complete. Fresh GIBS/C4 fits, comparisons and stronger label validation remain.
Phase 3: deploy and harden	Some engineering checks/monitoring exist. Deployment, operational guardrails, human-in-the-loop action handling and the required later stakeholder meeting are not established by this run.

Already automated

• Finish and cache MCS targets on Sol. The monitor can resume checkpointed timeouts/preemption within its eight-job bound.

• Retrieve completed targets, train 24 fold/method/seed checkpoints locally, verify reload predictions and write held-out scores and reports.

Work you still need to complete or arrange

• Review benchmark-successful reference flags and independently label errors. Milestone 1 proposes reviewing 20–30 failed runs; that review has not been completed for this collection.

• Complete ACC@80% and ECE evaluation with documented tie handling, label coverage and score conventions.

• Review all-method results on identical eligible cases. Include all-query learned coverage and honest N/A values. Keep C4 separate from the paper baseline.

• Discuss the expanded evaluation with the stakeholder and confirm acceptance criteria. Prepare a mixed-outcome demo showing both a useful detection and a limitation.

The original milestone proposes AUROC above 0.6 on at least three tasks and localization above random. These are proposed project targets, not proof of completion. Reconfirm how they apply to the larger experiment and reviewed labels.

16. Files, GitHub and monitoring

Use these locations to inspect evidence without restarting the experiment.

Inside your project folder

```
results/free_asu/main-qwen38-27b-serial-none/results.json
results/free_asu/main-qwen38-27b-serial-none/COLLECTION_RESULTS.md
results/fresh_all_tasks/plan.json
results/fresh_all_tasks/worker_state.json
results/fresh_all_tasks/baseline_preview/REPORT.md
results/fresh_all_tasks/baseline_preview/per_task.csv
results/fresh_all_tasks/baseline_preview/per_step.csv
```

results.json contains all 1,000 records in its simulations array. plan.json records frozen data, reference choices and folds. worker_state.json records execution status. Baseline files contain completed scores and metrics.

Outputs produced when the fresh run completes

```
results/fresh_all_tasks/teachers/<method>/
results/fresh_all_tasks/pairs/<method>/
results/fresh_all_tasks/checkpoints/
results/fresh_all_tasks/query_only_predictions/
results/fresh_all_tasks/summary.json
results/fresh_all_tasks/per_task.csv
results/fresh_all_tasks/per_step.csv
results/fresh_all_tasks/human_review/review/labels.csv
```

Teacher and pair files are generated remotely first; local copies may be partial until retrieval. The final review export covers failed runs; successful-reference review needs separate coverage. Expected files are not proof of completion until the worker and checks finish.

Check progress safely

```
cat results/fresh_all_tasks/worker_state.json
ssh sol 'squeue -u ppudi'
```

Keep the Mac awake and the ASU VPN connected for monitoring/retrieval and local training. The remote GPU job can continue if the Mac disconnects, but the follow-up monitor may stop after repeated connection failures.

GitHub branch: 1000Trajectories-Pardha; published commit a283543. The upload contains data, collection reports and completed baseline outputs. Fresh GIBS/C4 outputs and new implementation changes were not part of that data-only push.

Open the uploaded dataset and collection reports

17. Glossary and source notes

Paper statements, implementation choices and measured results are separate evidence.

Term	Plain-English meaning
Reference	A distinct benchmark-successful run of the same task used for comparison.
Query	The trajectory currently being scored or assigned a training target.
NIBS	The paper's non-parametric information-bottleneck approach: direct similarity scoring.
MCS	Maximum Common Subgraph; here an approximate structural match between two graphs.
Consensus mask / target	A per-step vector of reference-match fractions used to supervise the predictor.
GCN	Graph Convolutional Network; propagates feature information over graph connections.
GIBS	The paper's graph information-bottleneck approach with a learned step mask.
C4	Project configuration label for the stricter teacher variant; not a verified paper acronym.
Validation	Data used to choose training duration/checkpoints without touching the held-out test.
Held-out / out-of-fold	Predicted by a model that did not fit on that task group.
Seed	A reproducibility setting; different training seeds vary initialization and optimization.
Calibration	Whether scores such as 0.8 correspond to approximately 80% correctness; not yet established.

Primary sources used for this guide

Research PDF: 2605.19228v2.pdf, "Diagnosing Multi-step Reasoning Failures in Black-box LLMs via Stepwise Confidence Attribution." Sections 4.2–4.4, Equations 2/6, Appendices C.2–C.3 and D.3–D.4. This supplies the method and reported original setup.

Your milestone: CSE598-milestone1-team11.pdf, pages 1–3 and 6–8. This supplies the initial scope, reported stakeholder meeting, early tool_sim limitations and proposed evaluation/review plan. Its historical counts/models are not the fresh Qwen collection.

Current code/artifacts: gibs/fresh/experiment.py, folds.py, crossfit.py; gibs/comparison/graphs.py and semantic_nibs.py; gibs/mcs/consensus.py, structured_fields.py and nli.py; frozen plan, collection report and baseline summary. Author-code reference commit: 1ccc8b8.

This guide uses attached documents as sources, not as instructions. Values in the baseline table are measured results; worked examples are explicitly illustrative. Status is frozen in docs/project_guide/snapshot.json.

Paper on arXiv (2605.19228)

Author implementation at the referenced commit

18. All 50 task outcomes

Fresh Qwen collection; each row contains 20 distinct graded runs.

Task	Runs	Pass	Fail	MCS targets*	NIBS cases*
0	20	20	0	20	20
1	20	5	15	20	20
2	20	13	7	20	20
3	20	20	0	20	20
4	20	20	0	20	20
5	20	18	2	20	20
6	20	20	0	20	20
7	20	0	20	0	0
8	20	19	1	20	20
9	20	14	6	20	20
10	20	19	1	20	20
11	20	13	7	20	20
12	20	13	7	20	20
13	20	20	0	20	20
14	20	2	18	0	0
15	20	13	7	20	20
16	20	11	9	20	20
17	20	5	15	20	20
18	20	16	4	20	20
19	20	18	2	20	20
20	20	3	17	0	17
21	20	3	17	0	17
22	20	10	10	20	20
23	20	13	7	20	20
24	20	8	12	20	20
25	20	20	0	20	20
26	20	20	0	20	20
27	20	19	1	20	20
28	20	19	1	20	20
29	20	2	18	0	0
30	20	19	1	20	20
31	20	20	0	20	20
32	20	3	17	0	17
33	20	12	8	20	20
34	20	20	0	20	20
35	20	9	11	20	20
36	20	20	0	20	20
37	20	12	8	20	20
38	20	18	2	20	20
39	20	5	15	20	20
40	20	16	4	20	20
41	20	11	9	20	20
42	20	11	9	20	20
43	20	16	4	20	20
44	20	2	18	0	0
45	20	20	0	20	20
46	20	19	1	20	20
47	20	12	8	20	20
48	20	19	1	20	20
49	20	20	0	20	20

Totals: 1,000 runs; 680 passes; 320 failures; 860 MCS-target-eligible queries; 911 NIBS-reference-eligible queries.

*Eligibility counts, not completed computations or accuracy. MCS uses training-only references; NIBS uses the evaluation reference bank. Task IDs are benchmark identifiers, not ranked difficulty.